

پروژه نهایی درس سیستم عامل: LotteryVM

طراحی یک جایگزین صفحه بندی احتمالی

گروه درس سیستم عامل دکتر انتظاری ملکی

۲۶ آذر ۱۴۰۴

طراحان پروژه

میلاذ زارعی ملکی

طاها بیکلریان



دانشگاه علم و صنعت ایران

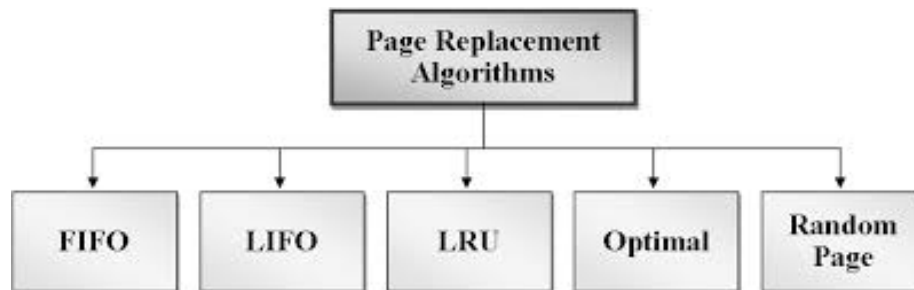
ترم ۴۰۴۱

فهرست مطالب

۲	نمای کلی پروژه (Project Overview)
۲	پیش‌زمینه نظری (Theoretical Background)
۳	حافظه مجازی و صفحه‌بندی (Virtual Memory & Paging)
۳	مکانیزم مبادله (The Swapping Mechanism)
۴	سیاست‌های جایگزینی صفحه (Page Replacement Policies)
۴	تایمرها و وقفه‌ها (Timers & Interrupts)
۵	کُد پایه: درک xv7-clean (The Base Code: Understanding xv7-clean)
۵	چه چیزی فراهم شده است؟ (What is provided?)
۵	فایل‌های کلیدی برای مطالعه (Key Files to Study)
۶	پاک‌سازی کد اولیه (The "Sanitization")
۶	الگوریتم: جایگزینی FIFO
	The Algorithm: Lottery-Based جایگزینی مبتنی بر قرعه‌کشی
۷	(Replacement)
۷	مفهوم: بلیط‌ها برابر است با حرارت (The Concept: Tickets = Heat)
۷	مکانیزم: لاتاری معکوس (The Mechanism: Inverse Lottery)
۷	پویشگر: پیر شدن صفحات (The Scanner: Aging)
	راهنمای پیاده‌سازی: مسئولیت شما (Implementation Guide: Your Responsibilities)
۸	(Responsibilities)
۸	فاز ۱: راه‌اندازی زیرساخت (Phase 1: Infrastructure Setup)
۸	فاز ۲: پویشگر (The Scanner - trap.c)
۹	فاز ۳: توزیع‌کننده (The Dealer - vm.c)
۹	فاز ۴: مدیریت حالات خاص (Handling Edge Cases)
۹	ارزیابی و محک‌زنی (Evaluation & Benchmarking)
۹	برنامه محک ('lotterytest.c') (The Benchmark: 'lotterytest.c')
۱۰	جمع‌آوری داده‌ها (Data Collection)
۱۰	ثبات توزیع بلیط‌ها (Debug Validation)
۱۰	اقلام تحویلی (Deliverables)
۱۰	کد منبع (Source Code)
۱۰	گزارش پروژه (Project Report)

نمای کلی پروژه (Project Overview)

در سیستم‌عامل‌های استاندارد، زمانی که حافظه با دسترسی تصادفی (RAM) پر می‌شود، سیستم‌عامل باید برخی از داده‌ها را به دیسک سخت منتقل کند تا فضای کافی برای پردازش-های فعال فراهم شود. به این فرآیند **مبادله** یا Swapping گفته می‌شود. تصمیم حیاتی‌ای که سیستم‌عامل باید بگیرد این است: کدام صفحه از حافظه باید به دیسک منتقل شود (بیرون انداخته شود)؟



شکل ۱: انواع سیاست‌های جایگزینی صفحه

اکثر سیستم‌ها از الگوریتم‌های قطعی مانند LRU (Least Recently Used) استفاده می‌کنند که کارآمد هستند، اما ردیابی دقیق آن‌ها می‌تواند از نظر محاسباتی پرهزینه باشد (زیرا نیاز به به‌روزرسانی لیست‌های پیوندی در هر دسترسی به حافظه دارد).

در این پروژه، شما یک **الگوریتم جایگزینی صفحه احتمالات** یا Probabilistic Page Replacement Algorithm را پیاده‌سازی خواهید کرد. به جای ردیابی دقیق ترتیب استفاده، شما مدیریت حافظه را مانند یک لاتاری در نظر خواهید گرفت:

۱. **بلیط‌ها (Tickets):** صفحات فعال، "بلیط‌های بخت‌آزمایی" جمع‌آوری می‌کنند.

۲. **قرعه‌کشی (The Draw):** زمانی که حافظه پر می‌شود، سیستم‌عامل یک قرعه‌کشی برگزار می‌کند. صفحاتی که بلیط‌های کمتری دارند (صفحات سرد یا Cold Pages)، از نظر آماری شانس بیشتری برای بیرون انداخته شدن دارند.

شما روی xv7 کار خواهید کرد؛ یک نسخه اصلاح‌شده از کرنل آموزشی MIT Xv6 که از قبل دارای پشتیبانی اولیه برای مکانیزم‌های swapping است. وظیفه شما پیاده‌سازی **سیاست** یا Policy است.

البته خب توجه داشته باشید که xv7 هم یک هسته‌ی آموزشی است و بنابراین ممکن است برخی از پیچیدگی‌ها و بهینه‌سازی‌های دنیای واقعی را نداشته باشد. با این حال، این هسته برای اهداف آموزشی ما کاملاً مناسب است و به ما امکان می‌دهد تا مفاهیم کلیدی مدیریت حافظه را به روشی قابل فهم و عملی یاد بگیریم.

پیش‌زمینه نظری (Theoretical Background)

برای موفقیت در این پروژه، درک عمیق تعامل بین سخت‌افزار و سیستم‌عامل ضروری است. کدی که شما خواهید نوشت، در تقاطع مدیریت حافظه (Memory Management) و مدیریت

وقفه‌ها (Interrupt Handling) قرار دارد.

حافظه مجازی و صفحه‌بندی (Virtual Memory & Paging)

سیستم‌عامل Xv6 (و نسخه مشتق شده xv7) از مکانیسم صفحه‌بندی (Paging) برای مدیریت حافظه استفاده می‌کند. این مکانیسم به سیستم‌عامل اجازه می‌دهد تا فضای آدرس مجازی (Virtual Address Space) که پردازنده‌ها می‌بینند را از آدرس‌های فیزیکی واقعی (Physical Addresses) در RAM جدا کند.

این نگاشت توسط سخت‌افزار پردازنده (واحد مدیریت حافظه یا MMU) و با استفاده از ساختاری به نام **جدول صفحه (Page Table)** انجام می‌شود.

- **مدخل جدول صفحه (Page Table Entry - PTE):** هر ورودی در جدول صفحه یک مقدار ۳۲ بیتی است که آدرس فیزیکی صفحه (۲۰ بیت بالا) و پرچم‌های کنترلی (۱۲ بیت پایین) را نگهداری می‌کند.

- **بیت حضور (PTE_P یا Present Bit):** اگر این بیت برابر با 1 باشد، به این معنی است که صفحه در RAM موجود است و پردازنده می‌تواند به آن دسترسی داشته باشد. اگر برابر با 0 باشد، هرگونه تلاش برای دسترسی به آن آدرس باعث ایجاد یک استثنا (Exception) به نام **خطای صفحه (Page Fault)** می‌شود.

- **بیت دسترسی (PTE_A یا Accessed Bit):** این بیت برای پروژه شما حیاتی است. سخت‌افزار پردازنده (در معماری x86) هر زمان که یک دستورالعمل حافظه (خواندن یا نوشتن) به آدرسی در این صفحه ارجاع دهد، به صورت خودکار این بیت را روی 1 تنظیم می‌کند. سیستم‌عامل می‌تواند این بیت را بخواند تا بفهمد آیا صفحه اخیراً "استفاده شده" (یا "داغ") است یا خیر، و همچنین می‌تواند آن را پاک کند (به 0 تنظیم کند) تا استفاده‌های آتی را ردیابی کند.

مکانیزم مبادله (The Swapping Mechanism)

مبادله یا Swapping مدیریت حافظه را به دو بخش مجزا تقسیم می‌کند:

۱. **مکانیزم (Mechanism):** چگونه ۴ کیلوبایت داده را از RAM به دیسک منتقل کنیم و چگونه آن را برگردانیم؟ این شامل درایورهای دیسک، تخصیص بلوک‌ها و مدیریت بافرهای ورودی/خروجی است.

۲. **سیاست (Policy):** چه کسی (کدام صفحه) را منتقل کنیم؟ این همان جایی است که الگوریتم‌های جایگزینی مانند FIFO، LRU یا Lottery وارد می‌شوند.

در xv7، مکانیزم برای شما فراهم شده است:

- زمانی که یک صفحه بیرون انداخته می‌شود (Evicted)، محتویات آن در ۸ سکتور متوالی دیسک نوشته می‌شود.

- بیت PTE_P پاک می‌شود تا نشان دهد صفحه دیگر در حافظه نیست.

- یک بیت جدید به نام PTE_SWAPPED (با مقدار 0x200) در PTE تنظیم می‌شود. این پرچم به سیستم‌عامل می‌گوید که "این صفحه معتبر است، اما فعلاً روی دیسک زندگی می‌کند".

- آدرس دیسک (شماره بلوک) در بیت‌های بالای PTE ذخیره می‌شود تا در صورت نیاز به بازگرداندن صفحه، سیستم بداند داده‌ها کجای دیسک هستند.

سیاست‌های جایگزینی صفحه (Page Replacement Policies)

زمانی که حافظه فیزیکی پر می‌شود و یک پردازنده نیاز به تخصیص حافظه جدید دارد (kalloc شکست می‌خورد)، سیستم عامل باید یک "قربانی" (Victim) را انتخاب کند تا فضا آزاد شود. انتخاب قربانی مناسب تأثیر چشمگیری بر کارایی سیستم دارد.

- **FIFO (First-In, First-Out):** ساده‌ترین سیاست است. سیستم صفحه‌ای را که زودتر از همه وارد حافظه شده است بیرون می‌اندازد. مشکل اینجاست که شاید آن صفحه قدیمی، حاوی کدهای مهم و پرکاربرد کرنل یا کتابخانه باشد. شما این سیاست را نیاز هست اضافه کنید.

- **LRU (Least Recently Used):** صفحه‌ای که برای طولانی‌ترین مدت استفاده نشده است را بیرون می‌اندازد. این الگوریتم بهینه‌تر است زیرا فرض می‌کند صفحاتی که اخیراً استفاده نشده‌اند، در آینده نزدیک هم استفاده نخواهند شد. پیاده‌سازی دقیق LRU پرهزینه است زیرا نیاز به تغییر لیست‌های پیوندی در هر دسترسی به حافظه دارد.

- **لاتاری معکوس (Inverse Lottery):** این روشی است که شما پیاده‌سازی خواهید کرد. به جای ردیابی قطعی، ما به صفحات بر اساس فعالیتشان "شال" یا "بلیط" می‌دهیم. صفحاتی که بلیط بیشتری دارند (فعال‌تر هستند)، احتمال بیرون انداخته شدنشان کمتر است. این روش تقریبی از LRU را با سربار کمتر و انعطاف‌پذیری بیشتر ارائه می‌دهد.

تایمرها و وقفه‌ها (Timers & Interrupts)

سیستم عامل زمان را با استفاده از یک تایمر سخت‌افزاری (مانند LAPIC) ردیابی می‌کند که به صورت دوره‌ای (مثلاً هر ۱۰ میلی‌ثانیه) یک وقفه (Interrupt) تولید می‌کند. این "تپش قلب" سیستم عامل است.

در Xv6، این وقفه باعث می‌شود کنترل از برنامه کاربر به کرنل منتقل شود (در تابع trap در فایل 'trap.c'). معمولاً از این وقفه برای تعویض پردازنده‌ها (Scheduling) استفاده می‌شود تا توهم همزمانی ایجاد شود.

در این پروژه، شما از این وقفه برای **پویشگر (Scanner)** خود استفاده خواهید کرد. شما نمی‌توانید در هر لحظه که دسترسی به حافظه رخ می‌دهد، بلیط‌ها را به روزرسانی کنید (این کار سیستم را بسیار کند می‌کند). در عوض، شما به صورت دوره‌ای (مثلاً در هر تیک تایمر) صفحات فعال را بررسی می‌کنید، بیت PTE_A آن‌ها را می‌خوانید و بر اساس آن بلیط‌ها را توزیع می‌کنید.

گد پایه: درک xv7-clean (Under-Base Code) (standing xv7-clean)

ما برای این پروژه، یک نسخه منشعب شده (Fork) از Xv6 به نام 'xv7-clean' را در اختیار شما قرار می‌دهیم. این نسخه دارای «لوله‌کشی» (Plumbing) لازم برای مبادله (Swapping) است، اما فاقد هوشمندی لازم برای انجام کارآمد آن می‌باشد. در واقع، زیرساخت‌ها آماده‌اند، اما مغز متفکر سیستم شما هستید.

رپوی پروژه در آدرس زیر قابل دسترس است:

<https://github.com/MiliAxe/xv7-clean>

چه چیزی فراهم شده است؟ (What is provided?)

- **تخصیص بلوک دیسک (Disk Block Allocation):** تابعی به نام 'balloc_page' در فایل 'fs.c' تعبیه شده است که ۸ سکتور متوالی دیسک (معادل ۴۰۹۶ بایت یا یک صفحه) را برای نگهداری محتویات یک صفحه حافظه رزرو می‌کند.
- **پرچم مبادله (Swap Flag):** یک تعریف جدید به نام PTE_SWAPPED (با مقدار 0x200) در فایل 'mmu.h' اضافه شده است. وقتی صفحه‌ای از حافظه بیرون انداخته می‌شود (Evicted)، بیت PTE_P صفر می‌شود و بیت PTE_SWAPPED یک می‌شود. این به سیستم می‌فهماند که صفحه نامعتبر نیست، بلکه در دیسک ذخیره شده است.
- **مدیریت خطای صفحه (Page Fault Handler):** در فایل 'trap.c'، کدی برای مدیریت خطای صفحه (T_PGFLT) وجود دارد. اگر آدرس مورد نظر دارای پرچم PTE_SWAPPED باشد، سیستم به طور خودکار توابعی را فراخوانی می‌کند تا آن صفحه را از دیسک به حافظه بازگرداند.

فایل‌های کلیدی برای مطالعه (Key Files to Study)

شما باید با فایل‌های زیر آشنا شوید تا بتوانید پروژه را انجام دهید:

فایل (File)	اهمیت و ارتباط (Relevance)
'kernel/mmu.h'	تعاریف بیت‌های جدول صفحه (PTE_A)، PTE_SWAPPED. اینجا را چک کنید تا بفهمید هر بیت چه معنایی دارد.
'kernel/fs.c'	به تابع 'balloc_page()' نگاه کنید تا بفهمید فضای دیسک چگونه برای صفحات رزرو می‌شود.
'kernel/bio.c'	تابع 'write_page_to_disk()' عملیات ورودی/خروجی سطح پایین (Raw I/O) را انجام می‌دهد.

فایل (File)	اهمیت و ارتباط (Relevance)
'kernel/vm.c'	وظیفه اصلی: حاوی تابع <code>select_a_victim()</code> است. در حال حاضر، این یک تابع ساختگی (Dummy) است که فقط سیاست FIFO را اجرا می‌کند. شما باید این تابع را بازنویسی کنید.
'kernel/trap.c'	وظیفه اصلی: شما باید در اینجا به وقفه تایمر (Timer Interrupt) متصل شوید (Hook) تا «پویشگر» (Scanner) خود را اجرا کنید.
'kernel/kalloc.c'	مدیریت فریم‌های فیزیکی حافظه. احتمالاً نیاز خواهید داشت ساختار داده‌ای برای نگهداری بلیط‌ها در اینجا اضافه کنید.

پاک‌سازی کد اولیه ("Sanitization")

کد اولیه xv7-clean یک فضای آماده برای پیاده سازی الگوریتم های شما فراهم میکند. با این حال، برخی از بخش‌های آن ممکن است شامل کدهای اضافی یا پیچیدگی‌هایی باشد که برای اهداف آموزشی ما ضروری نیستند. بنابراین، توصیه می‌شود که قبل از شروع به پیاده‌سازی الگوریتم خود، کد را مرور کرده و هر بخشی که به نظر شما غیرضروری است را حذف کنید تا یک پایه کد تمیز و قابل فهم داشته باشید.

• **هدف شما:** پیاده سازی دو الگوریتم جایگزینی صفحه: FIFO و Lottery-Based Replacement.

• **توجه:** در کد داده شده به صورت پیش فرض سیاست Second Chance پیاده سازی شده که شما بایستی در مقابل این دو الگوریتم که در ادامه داده شده است مقایسه کنید. پس پیشنهاد میشود که حتما با استفاده از قابلیت های گیت حواستان باشد تا بتوانید بین نسخه های مختلف جابجا شوید.

الگوریتم: جایگزینی FIFO

الگوریتم FIFO یکی از ساده‌ترین و ابتدایی‌ترین سیاست‌های جایگزینی صفحه است. در این الگوریتم، صفحه‌ای که قدیمی‌ترین زمان ورود به حافظه را دارد، به عنوان قربانی انتخاب می‌شود و بیرون انداخته می‌شود.

برای پیاده‌سازی این الگوریتم در xv7-clean شما میتوانید با استفاده از یک اشاره گر ساده به اولین صفحه ای که وارد حافظه شده است اشاره کنید و هر بار که یک صفحه جدید وارد حافظه میشود این اشاره گر را به صفحه بعدی منتقل کنید. زمانی که به انتهای لیست رسیدید دوباره به ابتدای لیست باز میگردید.

الگوریتم: جایگزینی مبتنی بر قرعه‌کشی (The Algorithm: Lottery-Based Replacement)

الگوریتم شما از دو جزء اصلی تشکیل شده است: **پویشگر** (Scanner) که وظیفه پاداش دادن به صفحات فعال را دارد، و **توزیع‌کننده** (The Dealer) که وظیفه انتخاب قربانی برای بیرون انداختن را بر عهده دارد.

مفهوم: بلیط‌ها برابر است با حرارت (The Concept: Tickets = Heat)

- هر صفحه فیزیکی (Physical Page) در حافظه دارای یک شمارنده به نام 'tickets' است.
- **بلیط‌های بالا:** صفحه به طور مکرر مورد دسترسی قرار گرفته است. این صفحه «داغ» است و باید در RAM بماند.
- **بلیط‌های پایین:** صفحه بیکار بوده است. این صفحه «سرد» است و کاندیدای مناسبی برای انتقال به دیسک (Eviction) می‌باشد.

مکانیزم: لاتاری معکوس (The Mechanism: Inverse Lottery)

در یک قرعه‌کشی استاندارد (مانند زمان‌بندی پردازنده)، داشتن بلیط بیشتر شانس «برنده شدن» را افزایش می‌دهد. اما در اینجا، «برنده شدن» به معنای بیرون انداخته شدن از حافظه است (که اتفاق بدی است). بنابراین، ما نمی‌خواهیم صفحاتی که بلیط زیادی دارند انتخاب شوند.

- بنابراین، احتمال بیرون انداخته شدن (P_{evict}) باید **نسبت عکس** با تعداد بلیط‌ها (T) داشته باشد:

$$P_{evict} \propto \frac{1}{T}$$

- **نکته پیاده‌سازی:** از آنجا که محاسبات ممیز شناور (Floating-point) در سطح کرنل دشوار و کند است، شما باید این محاسبات را با اعداد صحیح شبیه‌سازی کنید. می‌توانید به هر صفحه یک «وزن اخراج» (Inverse Weight) اختصاص دهید که برابر است با تقسیم یک عدد ثابت بزرگ (C) بر تعداد بلیط‌ها:

$$E_i = \frac{C}{T_i}$$

(مثلاً $C = 10000$). هرچه بلیط‌ها (T_i) بیشتر باشد، وزن اخراج (E_i) کمتر خواهد بود و شانس انتخاب شدن کاهش می‌یابد.

پویشگر: پیر شدن صفحات (The Scanner: Aging)

ما نمی‌توانیم در هر دسترسی به حافظه تعداد بلیط‌ها را به‌روزرسانی کنیم، زیرا سربار بسیار زیادی ایجاد می‌کند. در عوض، از یک **پویشگر** (Scanner) استفاده می‌کنیم که به وقفه تایمر متصل است:

۱. هر N تیک تایمر (مثلاً هر ۱۰۰ میلی‌ثانیه)، سیستم عامل جدول صفحه پردازش‌های فعال را پیمایش می‌کند.
 ۲. بیت **دسترسی** (PTE_A) بررسی می‌شود.
 ۳. اگر 1 باشد: صفحه در بازه زمانی گذشته استفاده شده است. به آن پاداش دهید (مثلاً ۱۰ بلیط اضافه کنید) و مهم‌تر از همه، **بیت دسترسی را پاک کنید** (صفر کنید) تا سخت‌افزار بتواند استفاده‌های بعدی را مجدداً ثبت کند.
 ۴. اگر 0 باشد: صفحه بیکار بوده است. بلیط‌های آن را کاهش دهید (مثلاً ۱۰٪ از بلیط‌ها کم کنید یا یک مقدار ثابت کسر کنید) تا فرآیند «پیر شدن» (Aging) شبیه‌سازی شود.
- این مکانیزم تضمین می‌کند صفحاتی که زمانی داغ بوده‌اند اما دیگر استفاده نمی‌شوند، به تدریج بلیط‌های خود را از دست می‌دهند و در نهایت کاندیدای اخراج می‌شوند.

راهنمای پیاده‌سازی: مسئولیت شما (Implementation) (Guide: Your Responsibilities)

در این قسمت یک راهنمای گام‌به‌گام برای پیاده‌سازی الگوریتم جایگزینی صفحه مبتنی بر لاتاری ارائه شده است. البته که شما می‌توانید از این راهنما به عنوان نقطه شروع استفاده کنید و بسته به خلاقیت خود، بهبودها و تغییرات دلخواه را اعمال کنید.

فاز ۱: راه‌اندازی زیرساخت (Phase 1: Infrastructure Setup)

۱. یک ساختار داده (Struct) تعریف کنید تا فراداده (Metadata) هر صفحه فیزیکی را ردیابی کند (مثلاً 'struct core_map').
۲. این ساختار به فیلدهایی برای 'tickets' و احتمالاً 'allocated_time' (اگر می‌خواهید در شرایط برابر تصمیم‌گیری کنید) نیاز دارد.
۳. صفحات را در تابع 'kalloc' با یک مقدار پیش‌فرض بلیط (مثلاً ۱۰) مقداردهی اولیه کنید.

فاز ۲: پویشگر (The Scanner - trap.c)

۱. فایل 'trap.c' را تغییر دهید. بخش مربوط به وقفه تایمر ('IRQ_TIMER') را پیدا کنید.
۲. تابعی به نام 'update_tickets()' پیاده‌سازی کنید که جدول صفحه پردازش فعلی را پیمایش می‌کند.
۳. بیت PTE_A را بررسی کنید. شمارنده‌های بلیط خود را بر اساس آن به‌روزرسانی کنید.
۴. **حیاتی:** فراموش نکنید که پس از بررسی PTE_A، آن را پاک کنید تا سخت‌افزار بتواند دفعه بعد دوباره آن را تنظیم کند.

فاز ۳: توزیع کننده (The Dealer - vm.c)

۱. تابع `select_a_victim()` را در `vm.c` پیدا کنید.
۲. در حال حاضر، این تابع فقط صفحه بعدی در لیست را برمی گرداند (FIFO).
۳. این منطق را با لاتاری معکوس خود جایگزین کنید:
• «مجموع وزن معکوس» تمام صفحات را محاسبه کنید: $W_{total} = \sum \frac{1}{T_i}$
• یک عدد تصادفی R بین 0 و W_{total} تولید کنید. (شما باید یک مولد عدد شبه تصادفی ساده پیاده سازی کنید).
• لیست صفحات را پیمایش کنید و وزن هر صفحه را از R کم کنید. زمانی که $R \leq 0$ شد، قربانی خود را پیدا کرده اید.

فاز ۴: مدیریت حالات خاص (Handling Edge Cases)

- اگر یک صفحه ۰ بلیط داشته باشد چه می شود؟ (خطر تقسیم بر صفر).
- اگر حافظه کاملاً پر باشد چه؟

ارزیابی و محک زنی (Evaluation & Benchmarking)

شما باید ثابت کنید که الگوریتم شما نه تنها کار می کند، بلکه از سیاست پایه (FIFO) که در حال حاضر در xv7-clean وجود دارد، بهتر عمل می کند. برای این کار، باید داده جمع آوری کنید و آن را مصورسازی نمایید.

برنامه محک ('lotterytest.c') (The Benchmark)

شما باید یک برنامه کاربر (User Program) به نام `lotterytest.c` بنویسید که رفتار زیر را شبیه سازی کند:

۱. **تخصیص حافظه:** مقدار زیادی حافظه (بیشتر از حافظه فیزیکی موجود) تخصیص دهید تا سیستم مجبور به Swap شود. برای راهنمایی گرفتن می توانید از تعدادی از برنامه هایی که خود xv7 ارائه می دهد استفاده کنید. برای مثال، `memtest1`، `memtest2`، `memtest3`. توجه کنید که از این برنامه ها باید استفاده کنید تا مطمئن شوید که الگوریتم شما پایدار است.
۲. **الگوی دسترسی (Access Pattern):** همه صفحات برابر نیستند. برنامه شما باید از قانون 90/10 پیروی کند:
 - ۹۰٪ دسترسی ها باید به ۱۰٪ از صفحات تخصیص یافته باشد (مجموعه داغ یا Hot Set).
 - ۱۰٪ دسترسی ها باید به ۹۰٪ باقی مانده باشد (مجموعه سرد یا Cold Set).
۳. **حلقه اجرا:** این دسترسی ها را در یک حلقه طولانی (مثلاً ۱۰۰۰ تکرار) انجام دهید.

جمع‌آوری داده‌ها (Data Collection)

برای رسم نمودار، نیاز به اعداد دارید.

۱. **فراخوانی سیستمی (System Call):** یک syscall جدید به نام 'sys_get_faults' اضافه کنید که تعداد کل خطاهای صفحه (Page Faults) پردازش فعلی را برگرداند.

۲. **چاپ در کنسول:** در برنامه 'lotterytest.c'، در هر ۱۰ تکرار حلقه، تعداد کل خطاها را چاپ کنید:

```
printf(1, "%d, %d\n", iteration, total_faults);
```

ثبات توزیع بلیط‌ها (Debug Validation)

برای اینکه به ما (و خودتان) ثابت کنید که سیستم توزیع بلیط کار می‌کند، یک مکانیزم چاپ موقت (Temporary Print) در تابع 'update_tickets' یا یک کلید میانبر کنسول (مانند 'CTRL+T') اضافه کنید که وضعیت ۱۰ صفحه اول را چاپ کند:

```
Page 0x...: Tickets=50, Accessed=1
```

این خروجی باید نشان دهد که صفحات «داغ»، بلیط‌های بیشتری نسبت به صفحات «سرد» دارند.

اقلام تحویلی (Deliverables)

شما باید موارد زیر را در یک فایل فشرده (Zip) تحویل دهید:

کد منبع (Source Code)

- کل پوشه xv7 تغییر یافته.
- کد باید کامپایل شود ('make qemu-nox') و برنامه 'lotterytest' باید بدون خطا اجرا شود.

گزارش پروژه (Project Report)

یک فایل PDF جامع شامل موارد زیر:

۱. طراحی و پیاده‌سازی (Design & Implementation):

- توضیحات کامل نحوه پیاده‌سازی الگوریتم‌های شما.
- توضیح دهید که ساختار داده «بلیط‌ها» را کجا ذخیره کردید؟
- فرمول دقیق شما برای «پاداش دادن» و «پیر کردن» صفحات چیست؟ (مثلاً: هر بار ۱۰+ بلیط، یا ضربدر ۲؟)
- نحوه پیاده‌سازی تولید عدد تصادفی (PRNG) را شرح دهید.

- بایستی الگوریتم شما کاملاً پایدار باشد. یعنی این که سیستم عامل نباید در حین اجرای برنامه کرش کند. توضیح دهید که چه اقداماتی برای اطمینان از این موضوع انجام دادید.

۲. نتایج تجربی (Experimental Results):

- یک نمودار خطی (Line Chart) رسم کنید که محور افقی آن «زمان» (تعداد دورهای حلقه) و محور عمودی آن «مجموع خطاهای صفحه» (Cumulative Page Faults) باشد.

- در این نمودار سه خط باید داشته باشید:

۱. الگوریتم FIFO پایه.

۲. الگوریتم جایگزینی صفحه مبتنی بر لاتاری شما.

۳. الگوریتم Second Chance که در کد اولیه وجود دارد.

- راهنمایی می‌توانید خروجی برنامه 'lotterytest' را در یک فایل متنی ذخیره کنید و سپس با استفاده از ابزارهایی مانند Excel یا Python (Matplotlib) نمودار را رسم کنید.

- نکته: در الگوریتم لاتاری که از یک مولد عدد تصادفی استفاده می‌کنید، ممکن است نتایج در هر اجرا کمی متفاوت باشد. برای اطمینان از صحت نتایج، پیشنهاد می‌شود که هر الگوریتم را حداقل ۵ بار اجرا کنید و میانگین نتایج را برای رسم نمودار استفاده کنید.

۳. تحلیل (Analysis):

- کدام الگوریتم شما بهترین عملکرد را داشت و چرا؟
- آیا نتایج با انتظارات شما مطابقت داشت؟
- آیا سربار (Overhead) اسکن کردن صفحات باعث کندی محسوس سیستم شد؟
- یک اسکرین‌شات از خروجی دیباگ (بخش Debug Validation) قرار دهید که نشان دهد صفحات داغ واقعاً بلیط بیشتری دارند.